Лабораторна робота 3

Автоматичне розпізнавання та переклад мовлення на базі Whisper

Глущенко Владислава, Григор'єв Андрій, Савчук Володимир

Квітень 2026

0. Зміст

1 Завдання **2**
1.1 Основне завдання — ASR 2
1.2 Додаткове завдання — AST (рівень 1) 2

2 Частина 1 — ASR на Toronto Dataset **2**
2.1 Опис датасету 2
2.2 Опис реалізації 2
2.3 Конфігурація тренування 3
2.4 Результати 3
2.5 Аналіз результатів 4

3 Частина 2 — AST на Nvidia Granary (додаткове) **4**
3.1 Опис датасету 4
3.2 Опис реалізації 4
3.3 Конфігурація тренування 5
3.4 Результати 5
3.5 Оцінка метрикою COMET 5
3.6 Аналіз результатів 6

4 Загальний підсумок **6**

1. Завдання

1.1. Основне завдання — ASR

Побудувати систему розпізнавання мовлення на Toronto Dataset:

1. Обрати доцільний претренований Whisper з HuggingFace.
2. Імплементувати логіку тренування за допомогою PyTorch Lightning.
3. Донавчити модель Whisper на частині датасету Toronto.
4. Не використовувати зафіксований тестовий набір для навчання або валідації; оцінити якість за метриками **WER** та **CER**.

1.2. Додаткове завдання — AST (рівень 1)

Реалізувати пайплайн для задачі автоматичного перекладу мовлення (Automatic Speech Translation): модель приймає аудіо однією мовою, а на виході генерує текст іншою. Підтвердити працездатність підходу на невеликому існуючому наборі даних.

2. Частина 1 — ASR на Toronto Dataset

2.1. Опис датасету

Toronto Dataset — зібрання україномовних аудіозаписів у форматі WAV (16 кГц), організованих у папки за мовцями/темами (`toronto_N`). Мітки зберігаються у файлі `labels.jsonl` у вигляді пар «шлях до файлу — текст транскрипції».

Після завантаження доступно **18 303 записи**; ще 10 929 файлів були відсутні на диску і автоматично пропускались. Стиль мовлення різноманітний: новинні фрагменти, побутові розмови, гумор, а також записи зі змішаним кодом (українська + англійська).

Розподіл на вибірки:

- **Тест** — 20 заздалегідь зафіксованих ID (наприклад, `toronto_27`, `toronto_46`, `toronto_9` тощо), виключно для фінального тестування.
- **Train / Val** — решта записів у співвідношенні 95 % / 5 % (випадковий shuffle).

2.2. Опис реалізації

Вибір моделі. Обрано **openai/whisper-tiny** — найменший варіант Whisper (37.8M параметрів). Вибір зумовлений обмеженнями обчислювальних ресурсів (Google Colab, Tesla T4) та достатньою підтримкою української мови. Процесор ініціалізується з параметрами `language="uk"` та `task="transcribe"`.

Препроцесинг. Кожен аудіофайл зводиться до монозвуку та ресемплюється до 16 кГц. Текстові мітки нормалізуються: переводяться у нижній регістр, видаляється пунктуація.

PyTorch Lightning. Клас `WhisperLightning` реалізує методи `training_step`, `validation_step` та `configure_optimizers` (AdamW, lr=10^{-5}). При ініціалізації

моделі знімається пригнічення кириличних токенів (`suppress_tokens = []`), що критично для коректної роботи з українською мовою. Тренування запускається через `pl.Trainer(max_epochs=5, accelerator="gpu")`.

DataCollator. Оскільки аудіо та тексти мають різні довжини, реалізовано кастомний колатор, який паддить вхідні ознаки та мітки до максимальної довжини у батчі. Мітки паддяться значенням `-100`, яке ігнорується при обчисленні loss.

2.3. Конфігурація тренування

Табл. 1: Гіперпараметри — ASR

Параметр	Значення
Модель	openai/whisper-tiny
Параметрів	37.8 M
Задача	transcribe, language=uk
Epochs	5
Batch size	8
Learning rate	10^{-5}
Optimizer	AdamW
GPU	Tesla T4

2.4. Результати

Табл. 2: Метрики на тестовій вибірці — ASR

Модель	WER ↓	CER ↓
whisper-tiny fine-tuned (5 epochs)	**0.592**	**0.361**

Табл. 3: Приклади розпізнавання

Еталон	**Передбачення**
це інстаграм телебачення торонто а це памятник свині а це телеграм телебачення торонто	це інстаграм телебачення торонто а це памятник свині а це телеграм телебачення торонто *(ідеально)*
слава ісу ви дивитесь програму 0 я є її ведучий майкл щур але я зараз у армії	слава ісу ви дивитесь програму 0 я її ведучий майкл щур але зараз у армі *(дрібні пропуски)*
з оголошення імпічменту ми починаємо процедуру імпічмента президенту	заголошення інпічменту ми починаємо процедуру інпічмента президенту *(«з»→«за», «імп»→«інп»)*
іде дощ і це так мало схоже на різдво	і до тож і цві так мало схож і нарістло *(сильні помилки)*

Приклади передбачень.

2.5. Аналіз результатів

Різниця між WER (59.2 %) і CER (36.1 %) на 23 п.п. свідчить, що більшість помилок — це *часткові* розбіжності на рівні символів, а не повністю хибні слова. Тобто модель правильно вловлює звуки, але помиляється в написанні («армії»→«армі», «імпічмент»→«іннічмент»).

Де модель справляється добре: чіткі фрази у стандартному або новинному стилі мовлення. Деякі речення відтворюються ідеально.

Де модель помиляється:

- Числа та нестандартні конструкції (*«класно1 класно2»*).
- Змішаний код: вставки англійських слів у переважно українське речення.
- Швидке або нечітке мовлення.

Причини обмежень: лише 5 епох тренування та невелика ємність whisper-tiny (37.8M параметрів) не дозволяють покрити все лексичне різноманіття датасету. Додатково, 10 929 відсутніх файлів зменшили фактичний обсяг навчальної вибірки.

3. Частина 2 — AST на Nvidia Granary (додаткове)

3.1. Опис датасету

Для задачі автоматичного перекладу мовлення використовувався датасет **nvidia/Granary**, підмножина `uk`, спліт `ast`. Він містить пари «україномовне аудіо — англійський текст перекладу» і завантажувався безпосередньо через HuggingFace `datasets`.

Розподіл: 80 % train / 10 % val / 10 % test (випадковий shuffle).

3.2. Опис реалізації

Підхід ідентичний до основного завдання з однією ключовою відмінністю: процесор і функція генерації ініціалізуються з параметром `task="translate"` замість `"transcribe"`. Кількість епох збільшена до 10. Датасет завантажується через `GranaryDataset`, який аналогічно до `TorontoDataset` обробляє аудіо та нормалізує текст.

3.3. Конфігурація тренування

Табл. 4: Гіперпараметри — AST

Параметр	Значення
Модель	openai/whisper-tiny
Задача	translate, language=uk
Epochs	10
Batch size	8
Learning rate	10^{-5}
Optimizer	AdamW
GPU	Tesla T4

3.4. Результати

Табл. 5: Метрики на тестовій вибірці — AST

Модель	WER ↓	CER ↓
whisper-tiny fine-tuned (10 epochs)	1.662	1.288

Табл. 6: Приклади перекладу

Еталон (англ.)	**Передбачення**
no just like fernhow we wont write either and well there are proper styles and there are improper ones	no just like the warhead we will not write about it and you are right stylists... *(семантично близько)*
the first time is different from what it was in the history of mankind they said absolutely nothing	the current time is being brought to the end of the story... *(значне відхилення)*
Hallucination loop: `the of the of the of the of the of the...`	

Приклади передбачень.

3.5. Оцінка метрикою COMET

Оскільки WER є ненадійною метрикою для перекладу (синоніми та відмінний порядок слів однаково штрафуються), додатково застосовано **COMET** (Crosslingual Optimized Metric for Evaluation of Translation) — сучасний стандарт оцінки машинного перекладу на базі нейромережевої семантичної схожості.

Для обчислення COMET готувались три файли:

- `src.txt` — оригінальний україномовний текст,
- `ref.txt` — еталонний англійський переклад,
- `hyp.txt` — переклад, згенерований моделлю.

Використовувалась модель `Unbabel/wmt22-comet-da`. Оцінка COMET знаходиться в діапазоні $[0, 1]$, де значення вище 0.85 відповідає якісному перекладу.

За результатами оцінки модель `whisper-tiny fine-tuned` отримала system score **COMET = 0.6515**.

3.6. Аналіз результатів

WER vs COMET. Незважаючи на катастрофічний WER = 166 %, COMET = 0.6515 демонструє помірну семантичну якість. Це підтверджує гіпотезу: WER сильно штрафує hallucination loops та незначні відхилення у формулюваннях, тоді як COMET оцінює смислову близькість і бачить, що частина перекладів є семантично прийнятною.

Hallucination loop залишається головною проблемою: деякі зразки генерують нескінченний повтор (`the of the of the...`), що обрушує WER/CER, але COMET також їх штрафує.

Що допомогло б:

- Параметр `no_repeat_ngram_size=3` при генерації — усуває цикли без зміни архітектури.
- Перехід на `whisper-small` або `whisper-medium`.
- Більша кількість епох або більший датасет.

4. Загальний підсумок

Усі пункти основного завдання виконано: обрано претренований `openai/whisper-tiny` з HuggingFace, логіку тренування реалізовано засобами PyTorch Lightning (`WhisperLightning`), модель донавчена протягом 5 епох на Toronto Dataset та протестована на 20 зафіксованих ID. Якість оцінено за метриками WER та CER через бібліотеку `evaluate`. Додаткове завдання першого рівня (AST) також виконано: реалізовано пайплайн перекладу на датасеті Granary uk/ast з оцінкою COMET = 0.6515.

Основне завдання виконано: модель донавчена, тестування проведено на зафіксованих ID, WER = 59.2 % та CER = 36.1 % відображають реальний рівень якості whisper-tiny після короткого файнтюну на різноманітному україномовному датасеті.

Додаткове завдання (AST) реалізоване на рівні пайплайну — підхід працездатний, але якість виявилась незадовільною через обмеження вибраної моделі.